

common_reference_t of reference_wrapper Should Be a Reference Type

Document #: P2655R1
Date: 2022-10-17
Project: Programming Language C++
Audience: SG9, LEWG
Reply-to: Hui Xie
<hui.xie1990@gmail.com>
S. Levent Yilmaz
<levent.yilmaz@gmail.com>

Contents

- 1 Revision History
 - 1.1 R1
 - 1.2 R0
- 2 Abstract
- 3 Motivation and Examples
- 4 Design
 - 4.1 Why Should the Result be `T&` and not `reference_wrapper<T>`
 - 4.2 Alternatives Considered
 - 4.2.1 Option 1: Support Exact Same Type with CV-Ref Variations
 - 4.2.2 Option 2: Treat `reference_wrapper<T>` as `T&`
 - 4.3 Supporting All Compatible Conversions (Option 3)
- 5 Implementation Experience
- 6 Wording
 - 6.1 Feature Test Macro
- 7 References

§ 1 Revision History

§ 1.1 R1

- Added reasons why the result should be `T&`
- Support `const` and `volatile`
- Support derive-base conversions

§ 1.2 R0

- Initial revision.

§ 2 Abstract

This paper proposes a fix that makes the `common_reference_t<T&, reference_wrapper<T>>` a reference type `T&`.

§ 3 Motivation and Examples

C++20 introduced the meta-programming utility `common_reference` 21.3.8.7 [meta.trans.other] in order to programmatically determine a common reference type to which one or more types can be converted or bounded.

The precise rules are rather convoluted, but roughly speaking, for given two non-reference types `X` and `Y`, `common_reference<X&, Y&>` is equivalent to the expression `decltype(false ? declval<X&>() : declval<Y&>())` provided it is valid. And if not, then user or a library is free to specialize the `basic_common_reference` trait for any given type(s). (Two such specializations are provided by the standard library, namely, for `std::pair` and `std::tuple` which map `common_reference` to their respective elements.) And if no such specialization exists, then the result is `common_type<X, Y>`.

The canonical use of `reference_wrapper<T>` is its being a surrogate for `T&`. So it might be surprising to find out the following:

```
int i = 1, j = 2;
std::reference_wrapper<int> jr = j; // ok - implicit constructor
int & ir = std::ref(i); // ok - implicit conversion
int & r = false ? i : std::ref(j); // error - conditional expression is ambiguous.
```

The reason for the error is not because `i` and `ref(j)`, an `int&` and a `reference_wrapper<int>`, are incompatible. It is because they are too compatible! Both types can be converted to one another, so the type of the ternary expression is ambiguous.

Hence, per the current rules of `common_reference`, given lack of a specialization, the evaluation falls back to `common_type<T, reference_wrapper<T>>`, whose `::type` is valid and equal to `T`. In other words, the `common_reference` type trait utility determines that the reference type to which both `T&` and a `reference_wrapper<T>` can bind is a prvalue `T`!

The authors believe this current determination logic for `common_reference` for an lvalue reference to a type `T` and its `reference_wrapper<T>` is merely an accident, and is incompatible with the canonical purpose of the `reference_wrapper`. The answer should have been `T&`. (There is no ambiguity with the `common_reference` of a prvalue `T` and `reference_wrapper`, since former is convertible to latter, but not vice versa).

This article proposes an update to the standard which would change the behavior of `common_reference` to evaluate as `T&` given `T&` and an a `reference_wrapper<T>`, commutatively. Any evolution to implicit conversion semantics of `reference_wrapper`, or of the ternary operator for that matter, is out of the question. Therefore, the authors propose to implement this change via providing a partial specialization of `basic_common_reference` trait.

Below are some motivating examples:

C++20	Proposed
<pre>static_assert(same_as< common_reference_t<int&, reference_wrapper<int>>, int>);</pre>	<pre>static_assert(same_as< common_reference_t<int&, reference_wrapper<int>>, int&>);</pre>

C++20	Proposed
<pre>static_assert(same_as< common_reference_t<int&, reference_wrapper<int>&>, int>);</pre>	<pre>static_assert(same_as< common_reference_t<int&, reference_wrapper<int>&>, int&>);</pre>
<pre>class MyClass { vector<vector<Foo>> foos_; Foo delimiter_; public: void f() { auto r = views::join_with(foos_, views::single(std::ref(delimiter_))); for (auto&& foo : r) { // foo is a temporary copy } } };</pre>	<pre>class MyClass { vector<vector<Foo>> foos_; Foo delimiter_; public: void f() { auto r = views::join_with(foos_, views::single(std::ref(delimiter_))); for (auto&& foo : r) { // foo is a reference to the // original element } } };</pre>
<pre>class MyClass { vector<Foo> foo1s_; Foo foo2_; public: void f() { auto r = views::concat(foo1s_, views::single(std::ref(foo2_))); for (auto&& foo : r) { // foo is a temporary copy } } };</pre>	<pre>class MyClass { vector<Foo> foo1s_; Foo foo2_; public: void f() { auto r = views::concat(foo1s_, views::single(std::ref(foo2_))); for (auto&& foo : r) { // foo is a reference to the // original element } } };</pre>

In the second and the third example, the user would like to use `views::join_with` and `views::concat` [P2542R2], respectively, with a range of `Foo`s and a single `Foo` for which they use a `reference_wrapper` to avoid copies. Both of the range adaptors rely on `common_reference_t` in their respective implementations (and specifications). As a consequence, the counter-intuitive behavior manifests as shown, where the resultant views' reference type is a prvalue `Foo`. There does not seem to be any way for the range adaptor implementations to account for such use cases in isolation.

§ 4 Design

§ 4.1 Why Should the Result be `T&` and not `reference_wrapper<T>`

As they can both be converted to each other, the result of `common_reference_t` can be either of them in theory. However, the authors believe that the users would expect the result to be `T&`. Given the following example,

```
auto r = views::concat(foos,
    views::single(std::ref(foo2_)));
for (auto&& foo : r) {
```

```
foo = anotherFoo;
}
```

If the result is `reference_wrapper<T>`, the assignment inside the for loop would simply rebind the `reference_wrapper` to a different instance. On the other hand, if the result is `T&`, the assignment would call the copy assignment operator of the original foos. The authors believe that the latter design is the intent of code and is the natural choice.

§ 4.2 Alternatives Considered

The following are some of the alternatives that considered originally. But later dropped in favor of the one discussed in the next section.

§ 4.2.1 Option 1: Support Exact Same Type with CV-Ref Variations

One option would be to provide customisations for only `reference_wrapper<T>` and `cv-ref T`. Note that this version is rather restrictive:

```
template <class T, class U, template <class> class TQual,
         template <class> class UQual>
    requires std::same_as<T, remove_cv_t<U>>
struct basic_common_reference<T, reference_wrapper<U>, TQual, UQual> {
    using type = common_reference_t<TQual<T>, U&>;
};

template <class T, class U, template <class> class TQual,
         template <class> class UQual>
    requires std::same_as<remove_cv_t<T>, U>
struct basic_common_reference<reference_wrapper<T>, U, TQual, UQual> {
    using type = common_reference_t<T&, UQual<U>>;
};
```

§ 4.2.2 Option 2: Treat `reference_wrapper<T>` as `T&`

This options completely treats `reference_wrapper<T>` as `T&` and delegates `common_reference<reference_wrapper<T>, U>` to the `common_reference<T&, U>`. Therefore, it would support any conversions (including derived-base conversion) that `T&` can do.

```
template <class T, class U, template <class> class TQual, template <class> class UQual>
    requires requires { typename common_reference<TQual<T>, U&>::type; }
struct basic_common_reference<T, reference_wrapper<U>, TQual, UQual> {
    using type = common_reference_t<TQual<T>, U&>;
};

template <class T, class U, template <class> class TQual, template <class> class UQual>
    requires requires { typename common_reference<T&, UQual<U>>::type; }
struct basic_common_reference<reference_wrapper<T>, U, TQual, UQual> {
    using type = common_reference_t<T&, UQual<U>>;
};
```

Immediately, it run into ambiguous specialisation problems for the following example

```
common_reference_t<reference_wrapper<int>, reference_wrapper<int>>;
```

A quick fix is to add another specialisation

```
template <class T, class U, template <class> class TQual, template <class> class UQual>
    requires requires { typename common_reference<T&, U&>::type; }
struct basic_common_reference<reference_wrapper<T>, reference_wrapper<U>, TQual, UQual>
{
    {
```

```
using type = common_reference_t<T&, U&>;  
};
```

However, this has some recursion problems.

```
common_reference_t<reference_wrapper<reference_wrapper<int>>,  
                  reference_wrapper<int>&>;
```

The user would expect the above expression to yield `reference_wrapper<int>&`. However it yields `int&` due to the recursion logic in the specialisation.

And even worse,

```
common_reference_t<reference_wrapper<reference_wrapper<int>>,  
                  int&>;
```

The above expression would also yield `int&` due to the recursion logic, even though the nested `reference_wrapper` is not convertible to `int&`.

The rational behind this option is that `reference_wrapper<T>` behaves exactly the same as `T&`. But does it?

There is conversion from `reference_wrapper<T>` to `T&`, and if the result requires another conversion, the language does not allow `reference_wrapper<T>` to be converted to the result.

This would cover majority of the use cases. However, this does not cover the derive-base conversions, i.e. `common_reference_t<reference_wrapper<Derived>, Base&>>`. This is a valid use case and the authors believe that it is important to support it.

§ 4.3 Supporting All Compatible Conversions (Option 3)

The above exposure can be extrapolated to any cv-qualified or other cross-type compatible conversions. That is, if `common_reference_t<U, V>` exists then `common_reference_t<reference_wrapper<U>, V>` and `common_reference_t<U, reference_wrapper<V>>` should also exist and be equal to it, *given* the only additional requirement that `reference_wrapper<U>` or `reference_wrapper<V>`, respectively, can be also implicitly converted to `common_reference_t<U, V>`. This statement only applies when the ternary conversion logic ambiguous.

The authors propose to support such behavior by allowing `basic_common_reference` specialization to delegate the result to that of the `common_reference_t` of the wrapped type with the other non-wrapper argument. Furthermore, impose additional constraints on this specialization to make sure that the `reference_wrapper` is convertible to this result.

In order to support commutativity, we need to introduce two separate specializations, and further constrain them to be mutually exclusive in order avoid ambiguity.

Finally, we have to explicitly disable the edge cases with nested `reference_wrappers` since, while `reference_wrapper<reference_wrapper<T>>` is not convertible to `T&`

§ 5 Implementation Experience

- The authors implemented the proposed wording below without any issue[[ours](#)].
- The authors also applied the proposed wording in LLVM's `libc++` and all `libc++` tests passed.

§ 6 Wording

Modify 22.10.2 [functional.syn] to add to the end of reference_wrapper section:

```
// [refwrap.common.ref] common_reference related specializations
template <class R, class T, template <class> class RQual, template <class> class TQual>
struct basic_common_reference<R, T, RQual, TQual>;

template <class T, class R, template <class> class TQual, template <class> class RQual>
struct basic_common_reference<T, R, TQual, RQual>;
```

Add the following subclause to 22.10.6 [refwrap]:

§ ??? common_reference related specializations [refwrap.common.ref]

```
template <class R, class T, template <class> class RQual, template <class> class TQual>
struct basic_common_reference<R, T, RQual, TQual> {
    using type = see below;
};
template <class T, class R, template <class> class TQual, template <class> class RQual>
struct basic_common_reference<T, R, TQual, RQual> {
    using type = see below;
};
```

1 The following are the *constraints*, and apply to both specializations verbatim:

- (1.1) — R is a cv-unqualified reference_wrapper<U> of some type U. [Note: T is any cv-unqualified type, possibly a reference_wrapper instance. -end note];
- (1.2) — common_reference_t<U&, TQual<T>> denotes a type. Let this type be Result.
- (1.3) — RQual<R> models convertible_to<Result>.
- (1.4) — Let the above constraints be expressed in the form CRW(R, RQual<R>, TQual<T>>) which evaluates to *true* when the conditions are satisfied. Then, CRW(T, TQual<T>, RQual<R>>) should evaluate to *false*. [Note: This final requirement provides mutual exclusion of the two specializations -end note];

The member typedef-name type denotes the type Result.

§ 6.1 Feature Test Macro

Add the following macro definition to 17.3.2 [version.syn], header <version> synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_common_reference_reference_wrapper_specialized 20XXXXL // also in
<functional>
```

§ 7 References

[ours] Hui Xie and S. Levent Yilmaz. A proof-of-concept implementation of common_reference_t for reference_wrapper.

https://github.com/huixie90/cpp_papers/tree/main/impl/ref_wrapper

[P2542R2] Hui Xie, S. Levent Yilmaz. 2022-05-11. views::concat.

<https://wg21.link/p2542r2>